

Cola de prioridad y tres productores

En las anteriores versiones nos hemos encargado de resolver el problema del productor-consumidor utilizando como estructura de datos compartida una cola FIFO, y posteriormente evitando las condiciones de carrera mediante el uso de mutexes. Ahora, nuestro objetivo será aumentar la dificultad, utilizando una cola de prioridad y tres productores.

Los cambios que suceden en este ejercicio son notables. Ya no tenemos un solo productor, si no que ahora hay exactamente 3, los cuales leerán cada uno de un archivo diferente. Además, la forma de organizar actualmente los elementos en el buffer será diferente. La estructura FIFO se verá alterada, cambiándola por una cola de prioridad en la cual el consumidor removerá y consumirá cada ítem basándose en que aquellos que inserte uno de los productores tendrán prioridad 1 y serán consumidos primero, los de otro, 2 y los del último 3. Además, el consumidor deberá guardar la suma de los distintos elementos, ordenada según la prioridad que estos tienen.

La compilación y ejecución del programa no presenta cambios con respecto a los anteriores ejercicios: compilamos con `gcc código.c [-o nombre_ejecutable] -lpthread`, estando los parámetros opcionales entre corchetes. Luego, ejecutaremos con `./nombre_ejecutable [archivo1] [archivo2] [archivo3]`. Importante destacar que los archivos de entrada proporcionados `prod1.txt`, `prod2.txt` y `prod3.txt` (u otros cualesquiera con los mismos nombres u otros pasados por parámetro en la ejecución) deben estar en la misma carpeta que el ejecutable.

Atendiendo al código, comenzaremos por mencionar los cambios generales. Actualmente, `Buffer` sólo presenta 2 elementos: un entero para el contador de elementos, y un array de `N PrioInt`. Este tipo de dato consiste en una estructura con un entero para el valor y un `unsigned int` para el nivel de prioridad que tiene. Además, creamos otra estructura, de nombre `ProdArg`, la cual incluye un `unsigned int` utilizado como identificador de prioridad del hilo y una cadena con el nombre para el archivo. Esto será pasado a los productores como argumento. Ahora, el `pthread_t` para el productor pasa a ser un array para los tres, que recibirán como prioridad el sucesor de su índice. La función `main` procederá como hasta ahora, pero terminará imprimiendo por separado las sumas de los tres productores y las tres sumas del consumidor según prioridad.

Atendiendo ahora específicamente al código del productor, no hay grandes cambios en su estructura, más allá de la adición de un tiempo de espera aleatorio después de cada lectura del archivo (función `produce_item`) y la modificación de un contador global que luego se mencionará su utilidad. Dónde se producen mayores cambios es en `insert_item`, que ahora deberá recibir un valor de prioridad que será utilizado para añadirlo en el buffer. Además, debido al funcionamiento como cola de prioridad, cada vez que se introduce un elemento, este debe situarse al final de todos los elementos de su correspondiente valor de prioridad. Como ejemplo, si tenemos una cola con espacio para 80 enteros, y los que tienen prioridad 1 se

encuentran entre los índices 0 y 12, los de valor 2 entre 13 y 28 y los de valor 3 entre 29 y 40, insertar un nuevo elemento con prioridad 3 sería simplemente insertar el valor en la última posición, pero si quisiéramos insertar uno de prioridad 2, habría que hacerlo en la posición 29, la cual ya tiene un elemento de prioridad 3, por lo que habría que desplazar todos los elementos de prioridad 3 una posición a la derecha. Si bien es cierto que se podrían utilizar 3 arrays diferentes según prioridad y hacer que el consumidor retire ítems de aquel con mayor valor de importancia (y sería realmente más eficiente), el hecho de que se use el mismo vector para todo nos da una mayor probabilidad de carrera crítica, ya que son 4 hilos que intentan modificar el mismo buffer, por lo que esto nos permite demostrar mejor la solidez de solucionar el problema con mutexes. Por último, fue modificada también la función `produce_item`, pidiendo ahora el id del hilo (su valor de prioridad) para utilizarlo de índice a la hora de sumar, colocando el resultado en el espacio adecuado.

Pasando ahora al consumidor, nos encontramos con una situación similar. No hay ningún cambio en la estructura, más allá de la creación de una semilla de aleatoriedad para una espera aleatoria, igual que en el anterior caso. En `remove_item` podemos notar otro cambio, al igual que en `insert_item`. El comportamiento se ha adaptado al de nuestra nueva cola de prioridad, por lo que siempre sacará elementos por la primera posición del array. Se supone que en una cola normal no debería ser necesario, pero en nuestro caso lo era debido al uso de un array circular. Por comodidad, se abandonó la estructura circular (eliminando los índices `inicio` y `fin`, trabajando únicamente con `count`) y se optó por reordenar los elementos para mantener la organización adecuada. Es específicamente por esto que se eliminaron los valores de inicio y fin del buffer. Una vez se lee el ítem, se mueve todo el array una posición a la izquierda, haciéndolo desaparecer. En `consume_item` nos encontraremos exactamente con el mismo cambio a la hora de realizar la suma, además de otra espera aleatoria para simular que la función tarda más tiempo de lo normal en consumir.

Un cambio general que aún no ha sido mencionado es el hecho de que ha sido cambiada la condición de salida del consumidor. En los ejercicios anteriores esto no era necesario, pues se terminaba al llegar a 80 iteraciones. Ahora es necesario por el mismo motivo que en el ejercicio de generalización del problema productor-consumidor para `n` y `m` productores y consumidores resuelto con semáforos hecho en la práctica anterior: los deadlocks o interbloqueos. Esto se produce a causa de que los 3 productores terminan, el buffer se encuentra vacío y el consumidor se queda esperando indefinidamente en la variable de condición, ya que ningún productor puede despertarlo. Es por ello por lo que se hace un contador de productores terminados, el cual cuando llega a 3, hará salir al productor del bucle. Como este contador es global, debe protegerse con mutex para evitar inconsistencias causadas por carreras críticas entre los productores a la hora de modificarlo. Esto lo hacemos mediante bloqueando el acceso y al terminar, mandando una señal de buffer no vacío a todos los hilos con `pthread_cond_broadcast` y desbloqueando posteriormente el mutex.

Atendiendo ahora a los resultados, es necesario tener en cuenta que la manera de interpretarlos ha cambiado. Ahora, cada productor tiene su suma independiente del resto, mientras que el consumidor tiene 3 sumas, una por cada nivel de prioridad, o expresado de otra manera, una por cada productor. Entonces, para comprobar que no se hayan producido carreras críticas, se debe observar que la suma de cada uno de los productores coincida con las sumas hechas por el consumidor. Como en este caso las regiones críticas están protegidas por mutex, podemos observar que efectivamente las sumas resultan iguales:

RESULTADOS:

Suma productores: 4363, 4304, 4307

Suma consumidores: 4363, 4304, 4307

En conclusión, cambiar el buffer a una cola de prioridad conlleva múltiples cambios que nos hacen elegir entre eficiencia (uso de 3 buffers), complejidad a la hora de programar (uso de montículos binarios) o crear un programa con más probabilidad de conflicto para demostrar la utilidad de los mutexes, que fue nuestro caso. Aun teniendo tres productores trabajando y un consumidor que lee los ítems producidos por estos según prioridad, apenas se necesitan cambios en el uso del mutex para evitar condiciones de carrera, siendo necesario únicamente evitar la presencia de deadlocks, el cual es el único cambio real en cuanto al uso del mutex.